



Python Task

Problem Statement

You are required to architect and build a modular, interactive command-line Railway Ticket Booking System in Python. The system will handle passenger intake, layered fare calculations, baggage weight checks, and promotional discounts to generate a final booking summary.

The system must be fully interactive at runtime. Hardcoding data into your main logic or writing a single monolithic script will result in disqualification.

Strict modularity is mandatory.

Available Routes

The following bidirectional routes are available in the system:

Route	Distance
New Delhi ↔ Mumbai	1,460 km
New Delhi ↔ Kolkata	1,525 km
New Delhi ↔ Chennai	2,200 km
New Delhi ↔ Hyderabad	1,670 km
Mumbai ↔ Kolkata	1,970 km
Mumbai ↔ Chennai	1,300 km
Mumbai ↔ Hyderabad	711 km

Kolkata ↔ Chennai	1,200 km
Kolkata ↔ Hyderabad	1,600 km
Chennai ↔ Hyderabad	633 km

Train Types & Travel Classes

Train Types

- Superfast — Average Speed: 120 km/hr
- Express — Average Speed: 110 km/hr
- Fast Passenger — Average Speed: 90 km/hr

Note: Travel time is calculated as Distance ÷ Average Speed of the selected train

Travel Classes

Class	Fare Multiplier	Free Baggage Allowance
Sleeper	1.0×	20 kg per passenger
AC 3-Tier	1.5×	30 kg per passenger
AC 2-Tier	2.0×	40 kg per passenger

The 6-Step Fare Calculation Logic:

The fare for each passenger must be calculated individually using the following steps, in this exact order.

Step 1 — Slab Fare

Add a flat base fare of INR 100, then apply the following distance-based slab rates:

- First 100 km → INR 1.00 per km
- Next 200 km (101 to 300 km) → INR 0.80 per km
- Beyond 300 km → INR 0.60 per km

Step 2 — Senior Citizen Discount

If the passenger is strictly above 60 years of age, apply a 40% discount to the Step 1 total.

Step 3 — Train Premium

Multiply the Step 2 result by the appropriate train premium:

- Fast Passenger → 1.0×
- Express → 1.25×
- Superfast → 1.50×

Step 4 — Class Premium

Multiply the Step 3 result by the class multiplier as defined in the Travel Classes table above.

Step 5 — Excess Baggage Fee

The system must prompt for the baggage weight of each individual passenger. If the declared weight exceeds the free allowance for their travel class, add a penalty of INR 15 for every extra kilogram.

Step 6 — Flat Surcharge

Add a final flat surcharge per passenger based on train type:

- Superfast → +INR 100
- Express → +INR 50
- Fast Passenger → +INR 0

Promo Code System & Final Checkout

After all individual passenger fares are calculated, sum them to produce a Subtotal. Before displaying the final bill, the system must prompt the user for an optional promo code.

Code	Discount Applied
ADG20	Deducts a flat 20% off the Subtotal.

WINTER500	Deducts a flat INR 500 off the Subtotal. The total must not drop below zero.
------------------	--

Only one promo code may be applied per booking. If an invalid code is entered, the system must notify the user and proceed to checkout without any discount.

Architectural & Evaluation Requirements

The evaluation weighs system design, control flow, and code hygiene as heavily as the correctness of the final output.

Mandatory File Structure

Your solution must be split across multiple files. The minimum required structure is:

- **routes.py** — All static route and distance data.
- **fare.py** — All mathematical operations and fare logic.
- **main.py** — The system driver and user-facing interface only.

You are strongly encouraged to create additional modules if doing so improves the overall architecture.

Separation of Concerns

`main.py` must serve only as a coordinator. No mathematical calculations or static data may reside in the main driver file.

Error Handling

The program must be completely crash-proof. It must gracefully handle all of the following without terminating abruptly:

- Invalid string inputs where a number is expected
- Out-of-range menu selections
- Negative or zero baggage weight values
- Any other logical edge cases

Code Quality

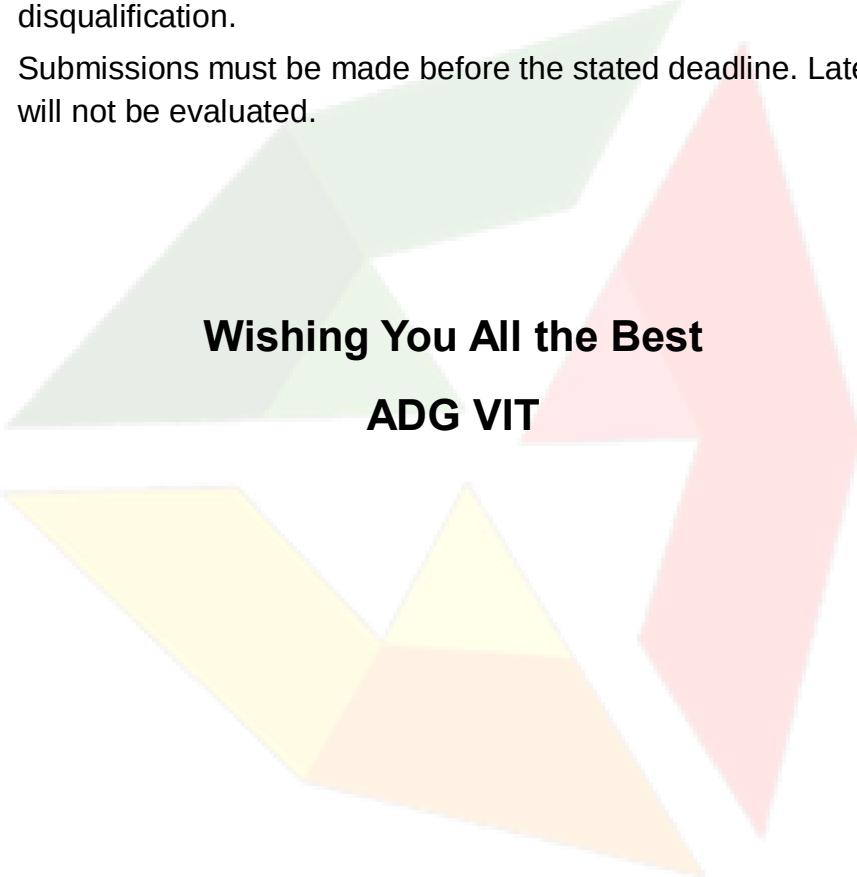
Clean and descriptive variable names, consistent formatting, and readable, decoupled functions are heavily weighted in the evaluation rubric.

Submission Guidelines

1. Submit your solution as a compressed archive (.zip) containing all source files.
2. Your repository must include a README.md with clear instructions for running the program.
3. All code must be original. Plagiarism will result in immediate disqualification.
4. Submissions must be made before the stated deadline. Late submissions will not be evaluated.

Wishing You All the Best

ADG VIT

A decorative graphic consisting of several overlapping, semi-transparent geometric shapes. There are two light green shapes at the top, a large light red shape on the right, and two light yellow shapes at the bottom. The shapes are arranged in a way that they appear to be part of a larger, abstract design.